RICE UNIVERSITY

Message Passing

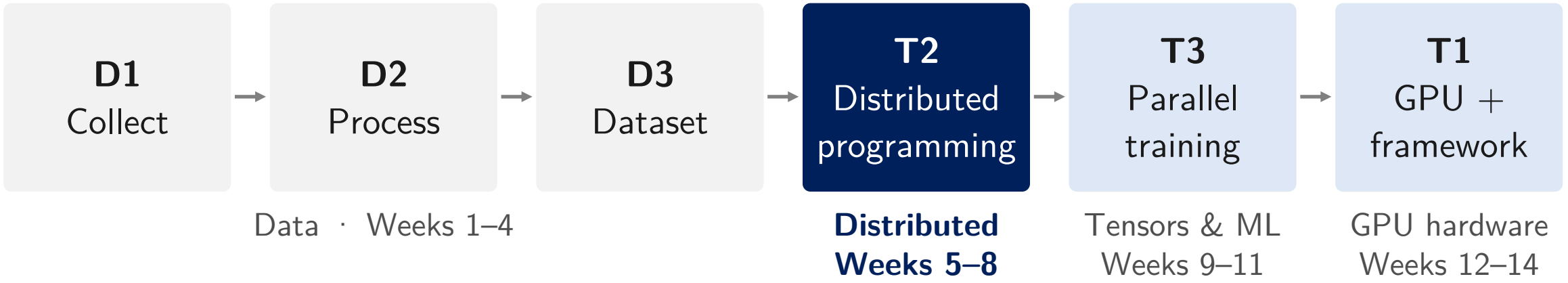
Processes, ranks, and messages: blocking, non-blocking, and what a message costs

Yuke Wang

Rice University

COMP 329 · Spring 2027 · Week 7, Lecture 1

Where we are in the pipeline



In our pipeline

Today: many processes that cooperate by **messages**. **D3 is due Friday**; T2 and Lab 4 are released after the next lecture.

Week 6: sockets, $\alpha + \beta n$ $\rightarrow$ today: messages $\rightarrow$ Week 7 L2: collectives $\rightarrow$ Week 8: all-reduce

Why one GPU is not enough: memory

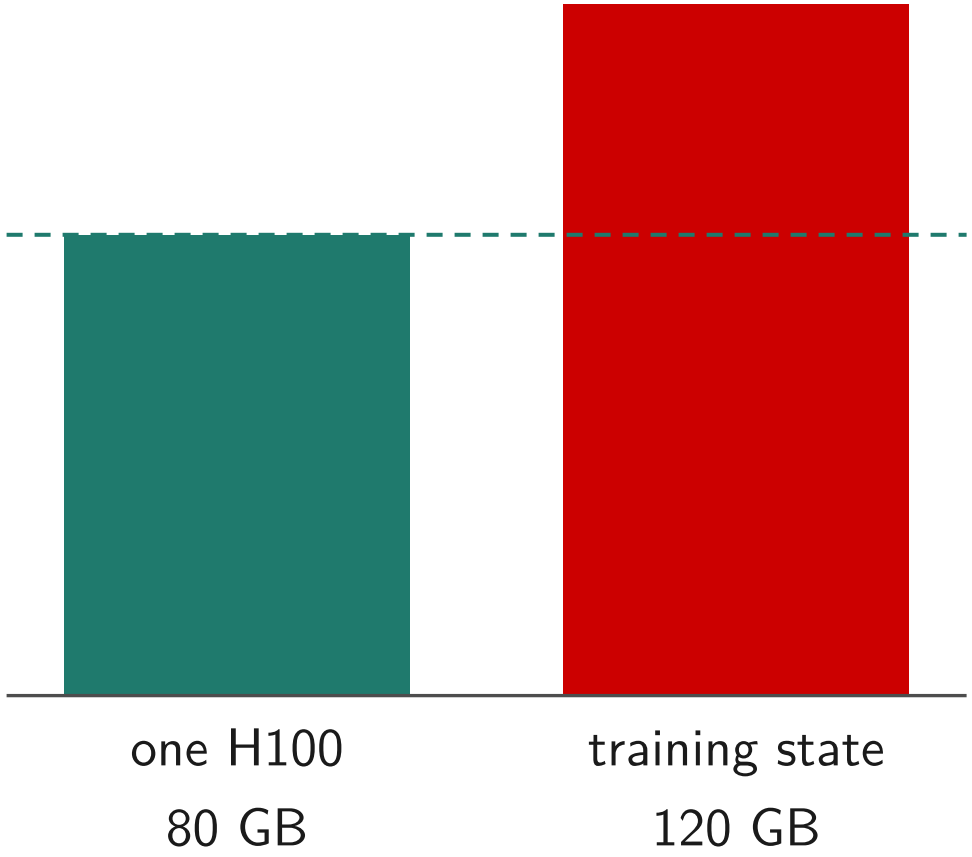
Can one 80 GB GPU train a 7.5-billion-parameter model?

parameter: one number that training learns.
Training with Adam keeps $\approx$ **16 bytes** per parameter.

$$7.5 \times 10^9 \times 16 \text{ bytes} = 120 \text{ GB}$$

Watch out

The training state alone exceeds one GPU.
Split it across many processes.



Source: Rajbhandari et al., ZeRO, SC 2020 (16 bytes per parameter); NVIDIA H100 datasheet (80 GB); 120 GB derived.

Why one GPU is not enough: time

And how long would the same model take to train? (assumptions in the notes)

$$\text{one GPU:} \quad \approx 7.2 \text{ years}$$

$$\text{1,024 GPUs, perfect scaling:} \quad \approx 2.6 \text{ days}$$

In our pipeline

"Perfect scaling" needs the GPUs to **communicate** cheaply every step — today's subject.

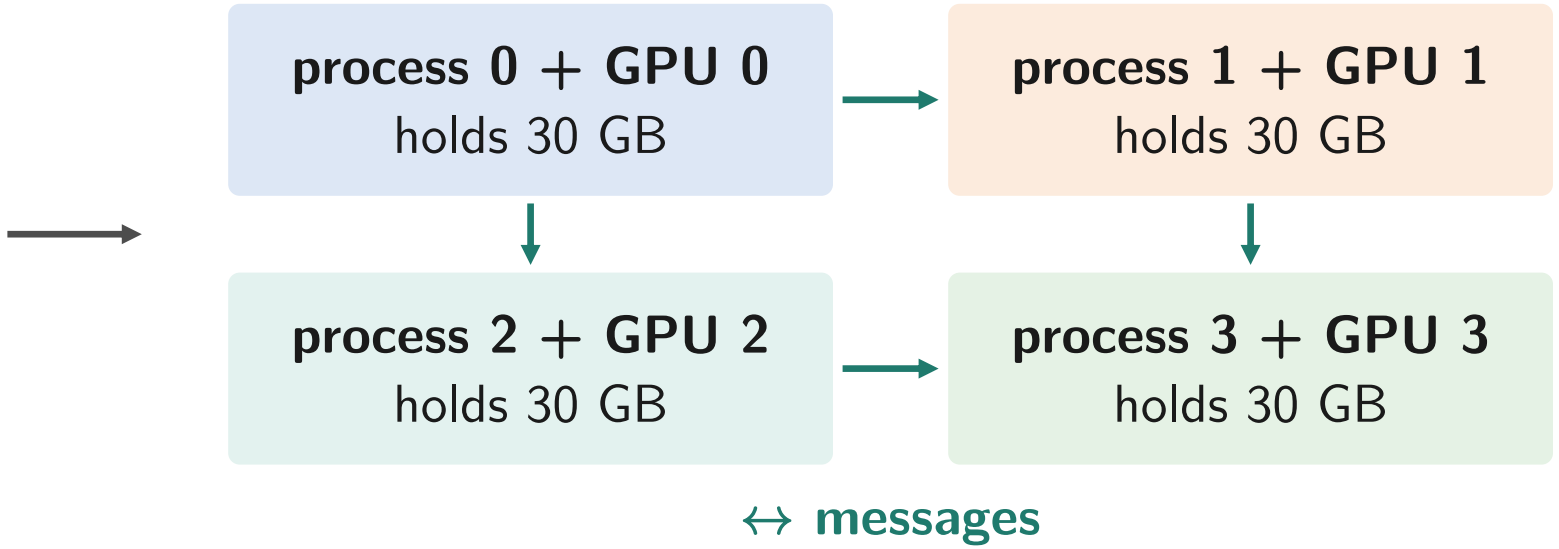
Source: Kaplan et al., 2020 ($C \approx 6ND$); H100 datasheet (BF16, dense); 40% utilization assumed; times derived.

What a distributed program does

process: one running copy of a program, with its own memory (and usually its own GPU).

message: data one process sends to another over a link.

1 process, 1 GPU
holds all 120 GB
$>$ 80 GB: does not fit



Key idea

Split the work and the state across processes; pay for it with **communication**.

Source: Rajbhandari et al., ZeRO, SC 2020; 30 GB per process derived.

Where distributed training shows up

System	Who talks to whom	In this course
MPI (mpi4py)	CPU processes, any machines	T2 all-reduce; Lab 4
NCCL (torch.distributed)	GPUs, in and across servers	T2 benchmark; Week 8
DDP, tensor, pipeline	many GPUs train one model	T3; Weeks 9–10

Llama 3 405B: trained on up to **16,384 H100s**; its training state alone needs the memory of $\approx$ **81 GPUs** (derived).

Why we need it

One GPU is too **small** to hold a large model's state and too **slow** to train it in time. Many processes, each with a GPU, cooperating by **messages**, fix both.

Source: Dubey et al., "The Llama 3 Herd of Models," 2024; ZeRO, SC 2020 (16 B/param); MPI 4.1 standard; NCCL docs.

Learning goals for today

- **Explain** why training needs many processes, and how each finds its **rank** and its **GPU**.
- **Send** and receive arrays, and explain when **blocking** calls **deadlock**.
- **Use** non-blocking calls to avoid deadlock and to **overlap** messages with work.
- **Fit** $T(n) = \alpha + n/\beta$ to measurements and **predict** a message time.

A puzzle to start

rank 0

1. send my array to rank 1
2. receive rank 1's array

rank 1

1. send my array to rank 0
2. receive rank 0's array

1 KB arrays: it finishes. 64 KB arrays: it hangs forever. Why?

Answer

A large send **waits for the matching receive**. Both ranks wait inside their send for the other's receive: a **deadlock**. Small sends are copied aside and return.

Source: one laptop run made for these slides: 2 processes, MPICH 5.0.2 shared memory, mpi4py 4.1.2.

Today's plan

1

Processes, ranks, and messages

How does each process know its job, and how do two processes exchange data?

2

Blocking and non-blocking

When does a call return, and how do we avoid waiting forever?

3

Measuring a message: α and β

How do measurements become two numbers that predict any message?

Build

Ping-pong and an α–β fit

Pairs: measure, fit, then predict a message you have not measured.

Part 1

Processes, ranks, and messages

How does each process know its job, and how do two processes exchange data?

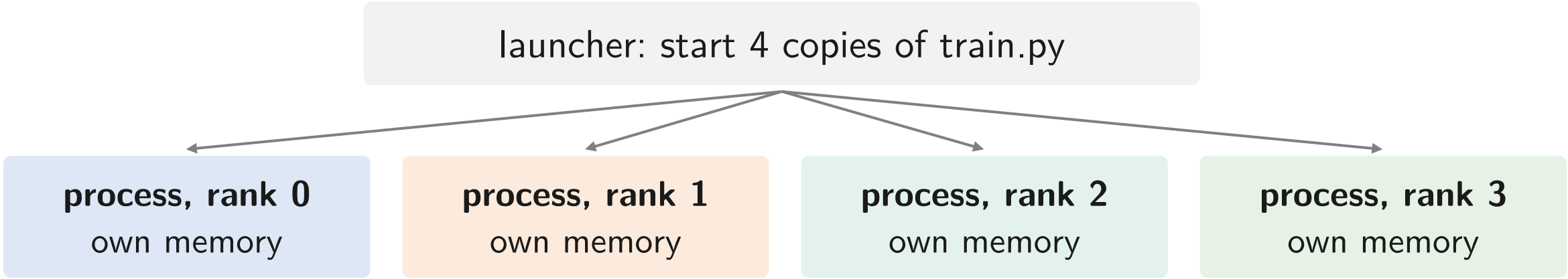
1 · Processes, ranks, and messages

2 · Blocking and non-blocking

3 · Measuring a message: α and β

Build · Ping-pong and an α–β fit

One program, many processes




```
rank = comm.Get_rank()      # which one am I?
p = comm.Get_size()         # how many of us?
mine = data[rank::p]        # my share of the data
```

rank: my number, 0 … p − 1.

p (world size): how many.

Key idea

Same code everywhere; behaviour differs only through **rank**. No shared memory: processes cooperate **only by messages**.

Rank and local rank

node 0 (one machine)

GPU 0	GPU 1	GPU 2	GPU 3
rank 0 local 0	**rank 1** local 1	**rank 2** local 2	**rank 3** local 3

node 1 (one machine)

GPU 0	GPU 1	GPU 2	GPU 3
rank 4 local 0	**rank 5** local 1	**rank 6** local 2	**rank 7** local 3

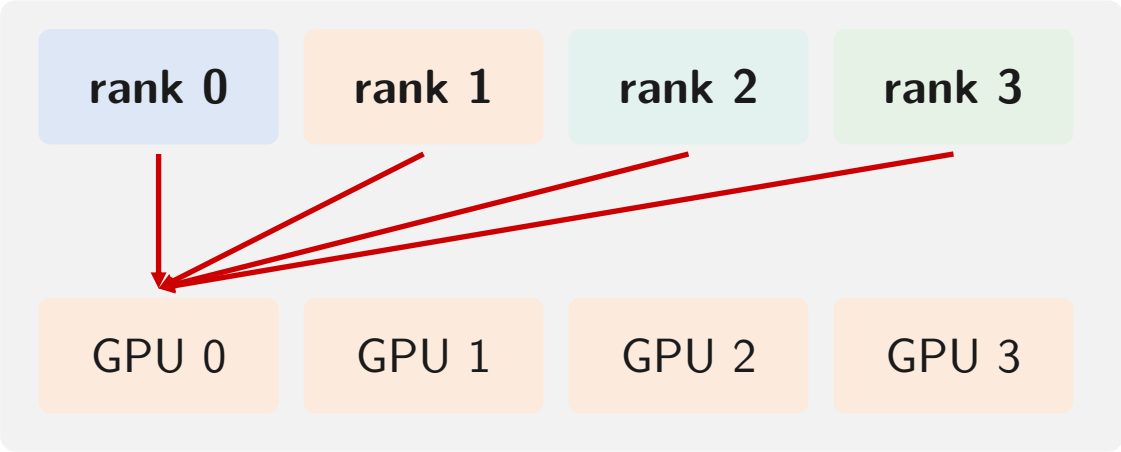
$$\text{rank} = \text{node} \times 4 + \text{local rank}, \qquad p = 2 \times 4 = 8$$

Key idea

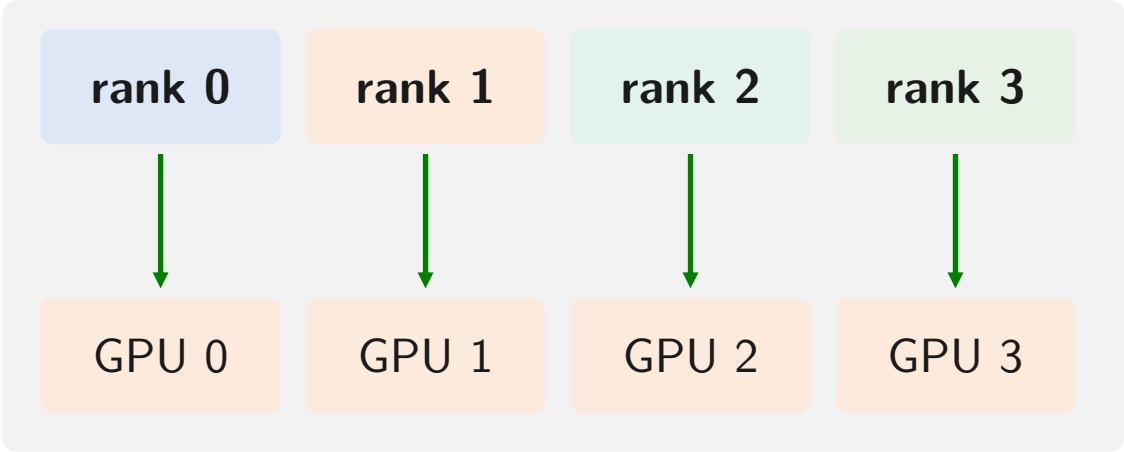
rank picks the work; **local rank** picks the GPU on this machine.

Mapping ranks to GPUs

default: everyone on GPU 0



after calling set_device




```
torch.cuda.set_device(local_rank)    # first!
```

one line, before any GPU work

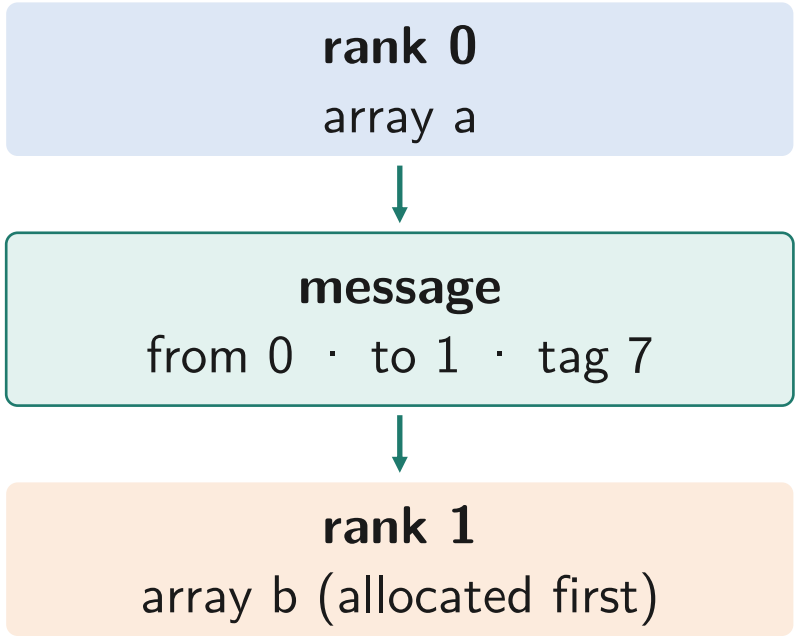
Watch out

Forget it and four processes share **GPU 0**: out of memory or slow, three GPUs idle.

Source: PyTorch Distributed Overview; torch.cuda.set_device documentation.

Send and receive

```
rank = comm.Get_rank()
if rank == 0:
    a = np.arange(1000, dtype=np.float64)
    comm.Send(a, dest=1, tag=7)
elif rank == 1:
    b = np.empty(1000, dtype=np.float64)
    comm.Recv(b, source=0, tag=7)
```



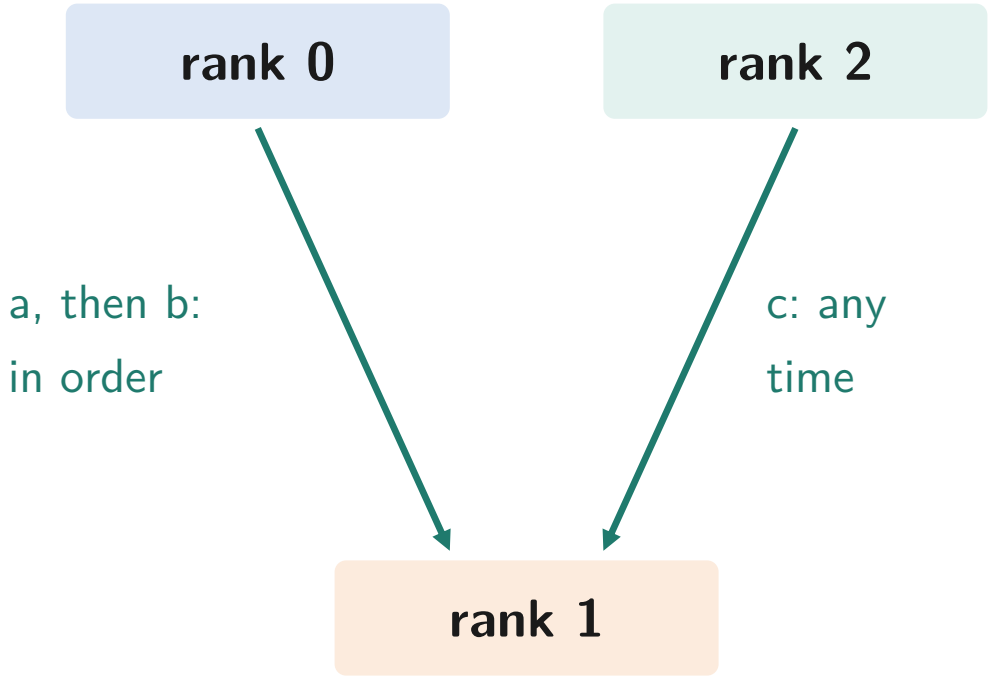
Key idea

tag: a label the sender picks. A receive takes the message whose **source and tag** match; the bytes land in **your** array.

Source: MPI Forum, MPI: A Message-Passing Interface Standard, Version 4.1 (2023); mpi4py documentation.

Which message does a receive take?

- A receive takes a message whose **source** and **tag** match.
- From **one** sender: messages arrive **in the order sent**.
- From **two** senders: **no** order is promised.



Key idea

Same sender: **ordered**. Different senders: use the **source** to tell them apart.

Source: MPI standard 4.1, Section 3.5.

Arrays or Python objects?

Methods	What they send	Tiny message
`Send` / `Recv`	an array's bytes, as they are	$\approx$ 0.9 µs
`send` / `recv`	any object, pickled (Week 6)	$\approx$ 7.7 µs

```
b = np.empty(1000)          # allocate first
comm.Recv(b, source=0)      # bytes land in b
```

In T2

For numbers, use **uppercase** methods on NumPy arrays: no pickling, no extra copies.

Source: mpi4py documentation; a separate laptop run (MPICH 5.0.2, mpi4py 4.1.2), one-way times.

Check yourself

Two minutes with a neighbour.

- 4 machines $\times$ 8 GPUs, one process per GPU: world size? Local rank of rank 13?
- A script forgets `set_device(local_rank)` on an 8-GPU machine. What do you observe?
- Rank 0 sends a, then b, to rank 1 (same tag). Which does rank 1 receive first?

Part 2

Blocking and non-blocking

When does a call return, and how do we avoid waiting forever?

1 · Processes, ranks, and messages

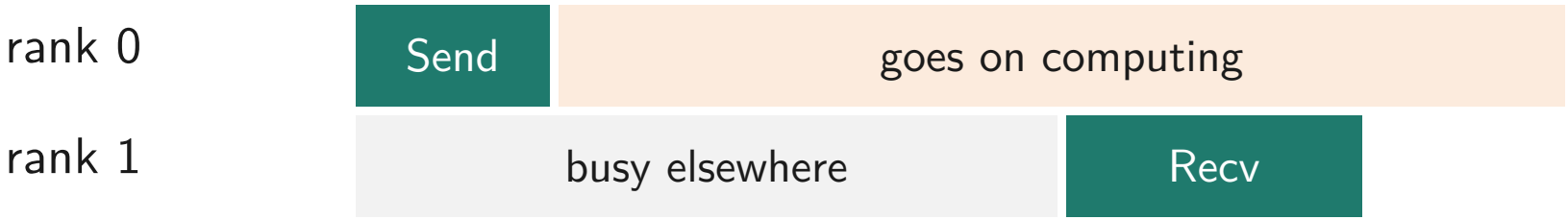
2 · Blocking and non-blocking

3 · Measuring a message: α and β

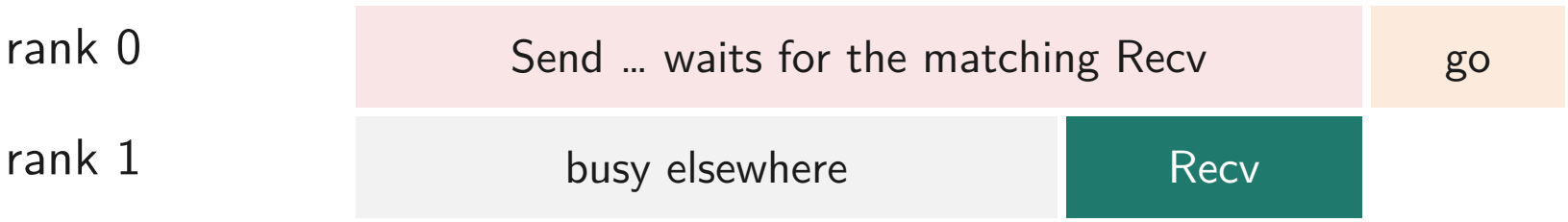
Build · Ping-pong and an α–β fit

What “blocking” means

small message



large message



Watch out

Send returns when its array **may be reused**. Large messages wait for the receiver — write code that works **either way**.

Source: MPI Forum, MPI: A Message-Passing Interface Standard, Version 4.1 (2023); mpi4py documentation.

Deadlock

rank 0

```
Send(a, dest=1)
Recv(b, source=1)
```

rank 1

```
Send(a, dest=0)
Recv(b, source=0)
```



each Send waits for a Recv

that comes after the other's Send

One laptop run

1 KB: **finished** (Send copied aside)

64 KB: **hung** (Send waited)

Watch out

Deadlock: every process waits for another, forever. Small tests can hide it.

Source: one laptop run made for these slides: 2 processes, MPICH 5.0.2 shared memory, mpi4py 4.1.2.

Two blocking fixes

1. Order by rank

```
if rank == 0:
    comm.Send(a, dest=1)
    comm.Recv(b, source=1)
else:
    comm.Recv(b, source=0)
    comm.Send(a, dest=0)
```

2. Let MPI pair them

```
other = 1 - rank
comm.Sendrecv(a, dest=other,
    recvbuf=b, source=other)
```

Check

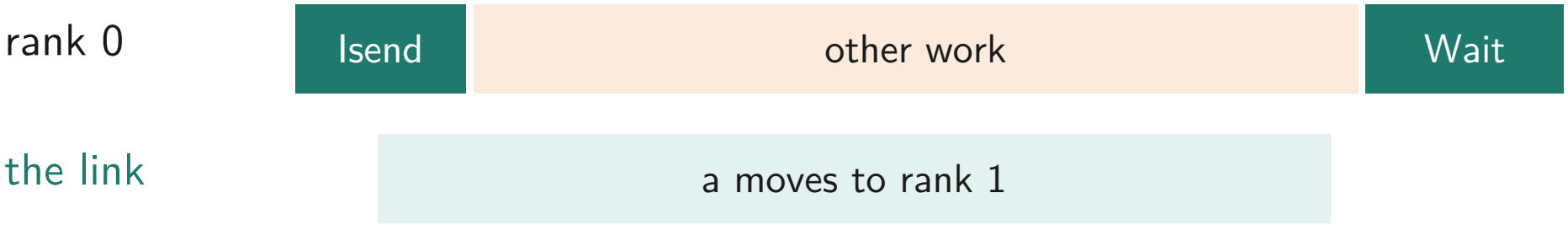
Both are correct for **any** size. But each process still **sits idle** while the bytes move.

Source: MPI Forum, MPI: A Message-Passing Interface Standard, Version 4.1 (2023); mpi4py documentation.

Non-blocking: start now, finish later

```
req = comm.Isend(a, dest=1)   # returns at once
other_work()                  # a is in flight
req.Wait()                    # done: a is free
```

request: a handle for a transfer that is still going on.



Key idea

Isend / Irecv start a transfer and return a **request** at once; **Wait** finishes it.

Source: MPI Forum, MPI: A Message-Passing Interface Standard, Version 4.1 (2023); mpi4py documentation.

When is it safe to touch the buffer?

```
req = comm.Isend(a, dest=1)
a[0] = 7            # BUG: a is still in flight
req.Wait()
```

Rule

After **Isend**: do not **change** the array until Wait.

After **Irecv**: do not **read** the array until Wait.

Watch out

Break the rule and rank 1 may get the old value, the new one, or a **mix** — and small tests often pass.

Source: MPI Forum, MPI: A Message-Passing Interface Standard, Version 4.1 (2023); mpi4py documentation.

Non-blocking removes the deadlock

```
other = 1 - rank
reqs = [comm.Isend(a, dest=other),
        comm.Irecv(b, source=other)]
MPI.Request.Waitall(reqs)
```

- Both Isends return at once.
- So both Irecvs are **posted**.
- Each send finds its receive.

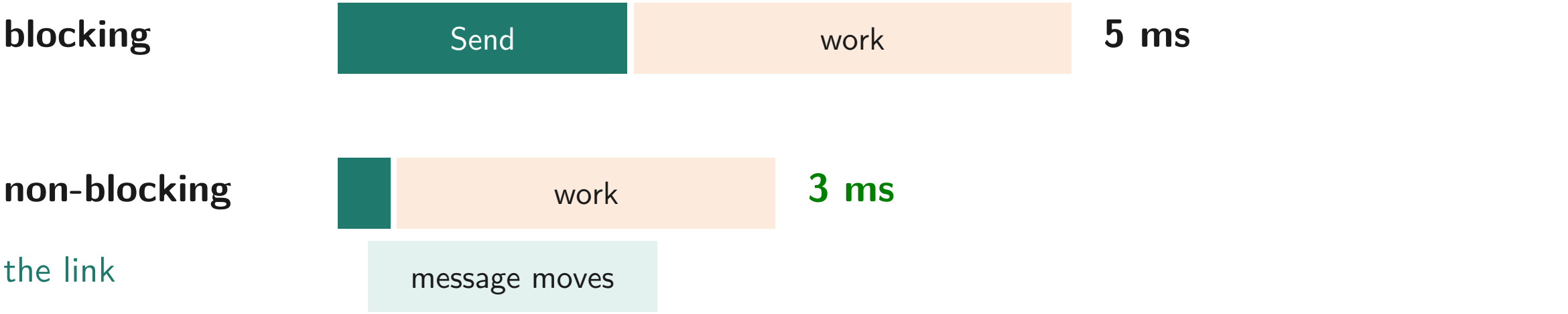
One laptop run, 8 MiB arrays (the blocking version hung at 64 KB): **finished**.

Check

No process waits inside a send, so there is **no cycle** — for any message size.

Source: one laptop run made for these slides: 2 processes, MPICH 5.0.2 shared memory, mpi4py 4.1.2.

Overlap: work while the message moves



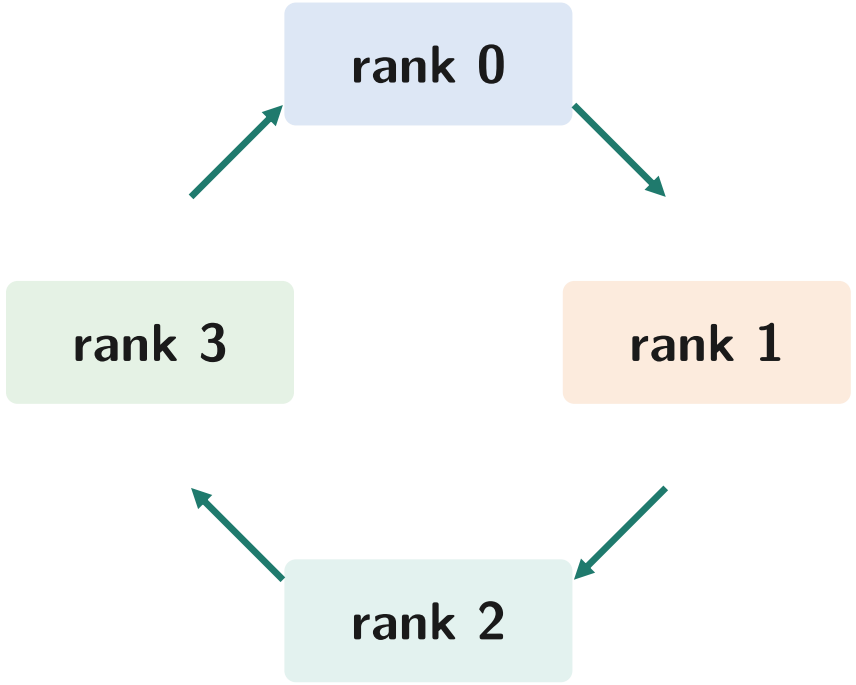
$$\text{blocking:} \quad T = t(\text{message}) + t(\text{work})$$

$$\text{overlapped:} \quad T \approx \max(t(\text{message}),\ t(\text{work}))$$

Key idea

Overlap **hides** communication behind independent work (training: Week 9).

Passing along a ring




```
right = (rank + 1) % p
left = (rank - 1) % p
reqs = [comm.Isend(a, dest=right),
        comm.Irecv(b, source=left)]
MPI.Request.Waitall(reqs)
```

Watch out

Blocking Send first: every rank waits for its right neighbour — a **cycle** of p ranks.

In our pipeline

Each rank sends right and receives left **at once**: the building block of T2's all-reduce.

Source: MPI Forum, MPI: A Message-Passing Interface Standard, Version 4.1 (2023); mpi4py documentation.

Check yourself

Two minutes with a neighbour.

- Both ranks call `Send` then `Recv` to each other. When does it work? Is it correct?
- After `Isend(a)`, rank 0 sets `a[0] = 7`, then calls `Wait`. What does rank 1 get?
- 4 ranks pass large arrays right with blocking `Send`, then `Recv`. What happens?

Part 3

Measuring a message: α and β

How do measurements become two numbers that predict any message?

1 · Processes, ranks, and messages

2 · Blocking and non-blocking

3 · Measuring a message: α and β

Build · Ping-pong and an α–β fit

Recap: the cost of one message (Week 6)

$$T(n) = \alpha + n / \beta$$

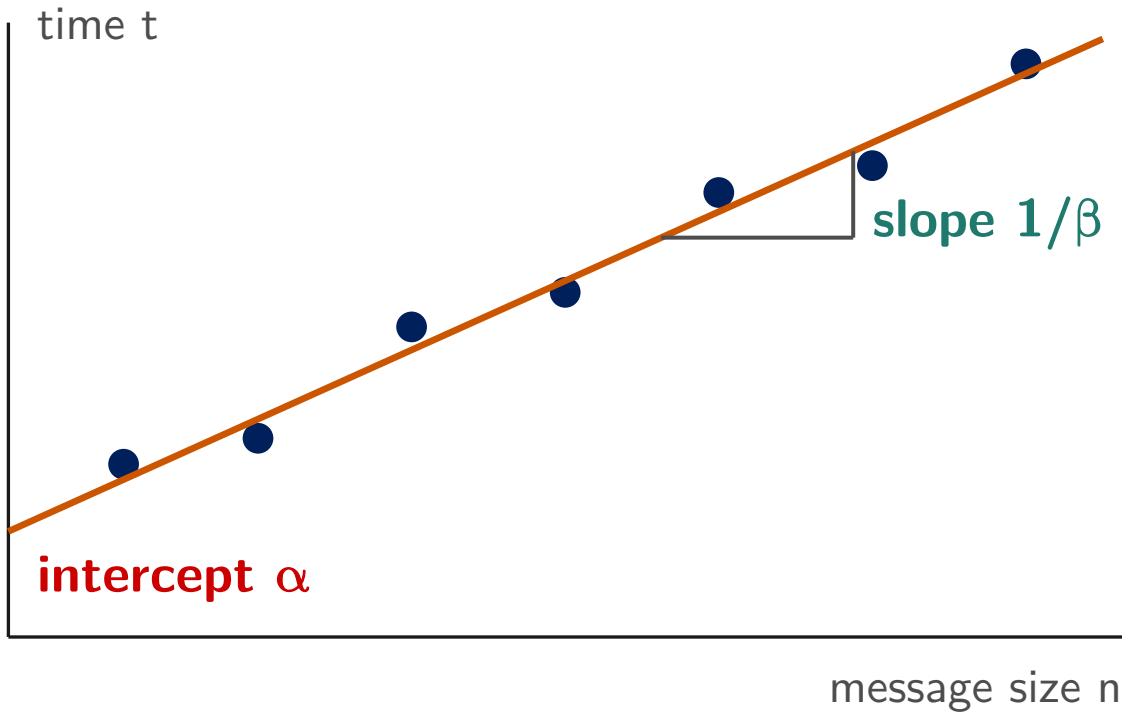
- α (latency): the fixed start-up cost of **every** message, in seconds.
- β (bandwidth): bytes per second once the data flow.
- Below $\mathbf{n_{1/2}} = \alpha\beta$ a message is latency-bound; above it, bandwidth-bound.

Week 6

Two numbers per link. Today: **measure** them, then **predict** a message we never timed.

Source: Hockney, Parallel Computing 20(3), 1994; Thakur, Rabenseifner & Gropp, IJHPCA 2005.

Fitting α and β: a straight line

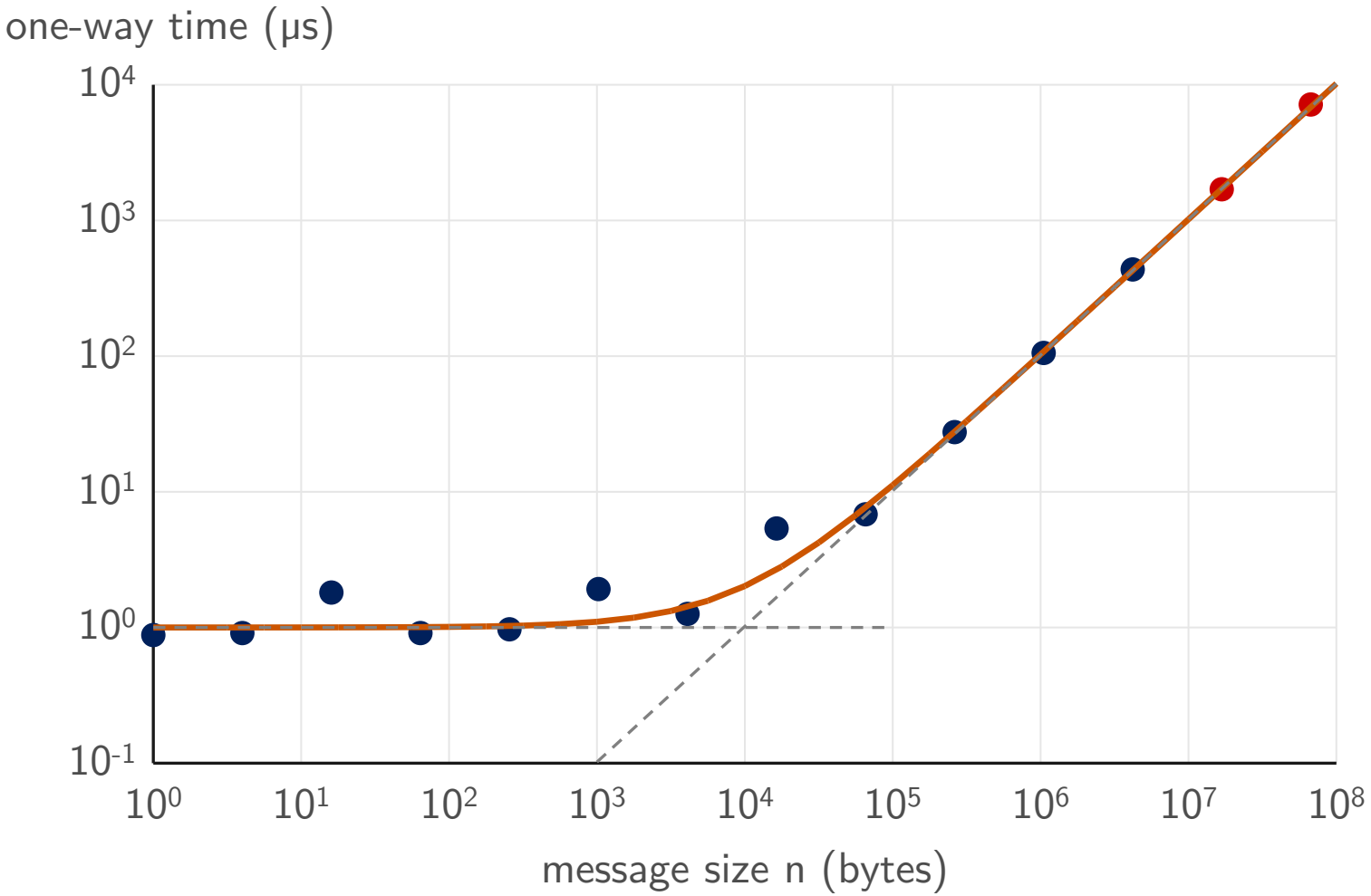


- Measure t at many sizes n.
- Fit the best straight line: **least squares**.
- Read off α and β.

Key idea

Fitting α and β is ordinary **linear regression** of time on size.

One laptop's ping-pong



Fit (blue points)

$\alpha \approx$ **1.0 µs**

$\beta \approx$ **9.8 GB/s**

blue: used in the fit

red: kept out

orange: fitted model

Source: one laptop run made for these slides: 2 processes, MPICH 5.0.2 shared memory, mpi4py 4.1.2.

Predict, then measure

64 MiB $= 2^{26}$ bytes — a size the fit never saw

$$\text{predicted:}\quad \alpha + 2^{26} \,/\, \beta \;\approx\; 6.8\ \text{ms}$$

$$\text{measured (one laptop run):}\quad 7.2\ \text{ms}$$

Check

About **4%** off: two numbers predict a message we never measured. Always predict **before** you measure.

Source: one laptop run made for these slides: 2 processes, MPICH 5.0.2 shared memory, mpi4py 4.1.2.

Check yourself

Two minutes with a neighbour.

- $\alpha = 5$ µs, $\beta = 25$ GB/s. What is n½? Is a 1 KB message latency- or bandwidth-bound?
- Same link: predict the time of a 100 MB message.
- A plain least-squares fit gives a **negative** α. What went wrong?

IN-CLASS BUILD

Ping-pong and an α–β fit

Pairs: measure, fit, then predict a message you have not measured.

Exercise 1: ping-pong

```
buf = np.zeros(n, dtype=np.uint8)   # n bytes
comm.Barrier(); t0 = MPI.Wtime()
for _ in range(reps):
    if rank == 0: comm.Send(buf, dest=1)
    comm.Recv(buf, source=1 - rank)
    if rank == 1: comm.Send(buf, dest=0)
t = (MPI.Wtime() - t0) / reps / 2   # one-way
```

DO

1. Predict t for 1 byte and for 16 MiB.
2. Run it on 2 processes, 1 byte to 16 MiB.
3. Save n and t to ping.txt.

ASK

Why can this loop never deadlock?

Source: mpi4py documentation; code run on one laptop (MPICH 5.0.2, mpi4py 4.1.2).

Exercise 2: fit, then predict

```
n, t = np.loadtxt("ping.txt", unpack=True)
A = np.column_stack([np.ones_like(n), n])
w = 1 / t                          # weight sizes equally
c, *_ = np.linalg.lstsq(A * w[:, None], t * w,
                        rcond=None)
alpha, beta = c[0], 1 / c[1]
print(alpha + 2**26 / beta)        # predict 64 MiB
```

DO

Write the 64 MiB prediction **before** measuring it. Then measure: how far off?

STRETCH

Run the ping-pong twice. Which changes more between runs, α or β?

Source: NumPy documentation, numpy.linalg.lstsq; code run on the laptop data of Part 3.

Solutions 1: the ping-pong

- **No deadlock**: rank 1 always receives **before** it sends.
- Shape: flat for small n ($\approx \alpha$), then growing in proportion to n ($\approx n/\beta$).
- One laptop run: 1 byte $\approx$ **0.9 µs**, 16 MiB $\approx$ **1.7 ms**.

In our pipeline

Your numbers will differ, especially the small ones; the **shape** will not.

Source: one laptop run made for these slides: 2 processes, MPICH 5.0.2 shared memory, mpi4py 4.1.2.

Solutions 2: the fit

$$\alpha \approx 1.0\ \mu\text{s}, \quad \beta \approx 9.8\ \text{GB/s}$$

64 MiB: predicted 6.8 ms, measured 7.2 ms

In our pipeline

T2 repeats this fit for **NCCL on GPUs** and for **two servers**.

Source: one laptop run made for these slides: 2 processes, MPICH 5.0.2 shared memory, mpi4py 4.1.2.

Key takeaways

- Large models need **many processes**: the training state alone exceeds one GPU.
- One program, p processes: **rank** picks the work, **local rank** picks the GPU.
- Blocking Send/Recv in the wrong order can **deadlock** — small tests may hide it.
- **Isend/Irecv** return at once: no deadlock, and **overlap**. Touch buffers only after **Wait**.
- $\mathbf{T(n) = \alpha + n/\beta}$: fit it by least squares, then **predict** before measuring.

Next time, and readings

When	What
Week 7, Lecture 2	Collective communication; T2 and Lab 4 released after it
Friday (this week)	D3 due
Week 8, Lecture 1	All-reduce: rings and trees

READINGS

MPI standard 4.1, Chapter 3 (point-to-point; blocking and nonblocking), or the mpi4py tutorial.

PyTorch documentation: "PyTorch Distributed Overview".